

Instituto Politécnico Nacional
Escuela Superior de Cómputo



Simulación de sistemas complejos mediante autómatas celulares neuronales.

Presentan
Angeles Lopez Erick Jesse[1]

Fecha: 17 de septiembre de 2025

[1]eangelesl1700@alumno.ipn.mx

Índice general

1. Introducción **4**

2. Marco teórico **5**

2.1. Autómatas celulares 5

2.1.1. Vecindad 6

2.1.2. Estados 6

2.1.3. Condiciones de frontera 9

2.1.4. Regla local 10

2.1.5. Clasificación 13

2.2. Redes neuronales 16

2.2.1. Neurona biológica 16

2.2.2. Neurona artificial 17

2.2.3. Perceptrón y perceptrón multicapa 21

2.2.4. Aprendizaje en redes neuronales 21

2.2.5. Redes neuronales 24

2.2.6. Arquitecturas 24

3. Estado del Arte **25**

3.1. Introducción 25
3.2. Simulación mediante autómatas celulares 26
3.3. Morfogénesis 28
3.3.1. NCA Isotrópicos 29
3.3.2. Autoclasificación 30
3.3.3. Representaciones tridimensionales 33
3.4. Mesh Neural Cellular Automata 36
3.5. Modelos generativos 37
3.6. Representaciones y aplicaciones adicionales 40

Referencias **43**

Capítulo 1

Introducción

Capítulo 2

Marco teórico

2.1. Autómatas celulares

Un autómata celular (CA, *Cellular Automata*), concebido por John von Neumann [1], es una colección de células que pueden adquirir un estado y que se agrupan en una red. El CA evoluciona en pasos de tiempo, donde cada célula cambia su estado en función de los estados de su vecindad. Durante la evolución, el CA cambia de una configuración a otra. Tradicionalmente, se considera una red regular de células, un conjunto finito de estados y evoluciones en pasos discretos de tiempo.

Definición 1. *Un autómata celular es una 4-tupla* $(\mathcal{L}, \mathcal{S}, \mathcal{N}, \mathcal{R})$ *donde:*

- $\mathcal{L} \subseteq \mathbb{Z}^D$ *es el espacio celular D-dimensional. Un conjunto de células se colocan en el espacio* $\mathcal{L}$ *para formar una red.*
- $\mathcal{S}$ *es un conjunto de estados. Cada célula solo puede adquirir un valor de un conjunto finito de diferentes valores:*

$$\sigma_{\vec{i}}(t) \in \mathcal{S} \equiv \{0, 1, 2, \ldots, k-1\},\ \forall \vec{i} \in \mathcal{L}$$

  Donde $\vec{i} \in \mathcal{L}$ *es una célula en el espacio celular,* $\sigma_{\vec{i}}(t)$ *es el estado de la i-ésima célula en el tiempo t y* $\mathcal{S}$ *es un anillo conmutativo finito*[1]*, generalmente definido por los enteros modulo k* $(\mathbb{Z}_k)$*.*

[1]Un anillo conmutativo finito es un conjunto finito dotado de dos operaciones internas —suma y multiplicación— donde la suma forma un grupo abeliano, la multiplicación es asociativa y conmutativa, y ambas operaciones están relacionadas por las leyes distributivas.

- $\mathcal{N} = (\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_m)$ *es el conjunto de vectores de desplazamiento que definen la vecindad de una célula en* $\mathcal{L}$*. Asi, los vecinos de las células ubicadas en* $\vec{i} \in \mathcal{L}$ *se encuentran en las posiciones* $\vec{i} + \vec{v}_j, \forall j \in \{1, 2, \ldots, m\}$
- $\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$ *es la regla local del autómata:*

$$\sigma_{\vec{i}}(t+1) = \mathcal{R}(\{\sigma_{\vec{i}+\vec{v}_j}\}_{j=1}^{m})$$

donde $\mathcal{R}$ *es la función de transición local que asigna el nuevo estado de una célula a partir de los estados de sus* m *vecinos.*

2.1.1. Vecindad

La vecindad de una célula está estrechamente relacionada con la dimensionalidad del CA. John von Neumann propone una vecindad para $\mathbb{Z}^2$ de cinco células formada por ella misma y las cuatro células ortogonales más cercanas como se observa en la figura 2.1a. Esta vecindad es expandida por Moore añadiendo los cuatro vecinos diagonales más cercanos, dando una vecindad de nueve elementos. Esta vecindad, representada por la figura 2.1b, fue utilizada por John Conway en el "Juego de la Vida" (GoL, *Game of Life*).

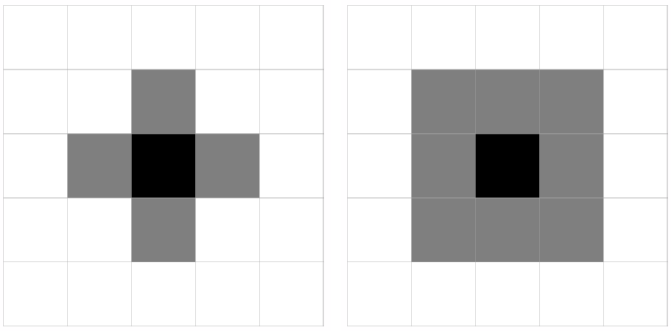
La figura 2.1c representa una vecindad de radio uno en un espacio unidimensional, es decir, todas aquellas células a una distancia menor o igual a uno, incluyéndose a su misma.. Fue utilizada por Stephen Wolfram para definir los Autómatas Celulares Elementales (ECA).

Las vecindades no se limitan a una cuadrícula regular; también pueden adoptar otras formas, como las vecindades hexagonales o triangulares ilustradas en las figuras 2.2a y 2.2b. Ademas, existen vecindades con comportamiento dinámico, como la vecindad de Margolus mostrada en la figura 2.2c, cuya configuración alterna entre dos disposiciones en cada paso evolutivo.

2.1.2. Estados

Cada célula adquiere un único estado s en cada paso de tiempo, donde s pertenece a un conjunto de estados $s \in \mathcal{S}$. El CA de John von Neumann contempla 29 estados, aunque algunos autores han propuesto variantes reducidas a cuatro estados, conservando su capacidad de autoreproducirse.

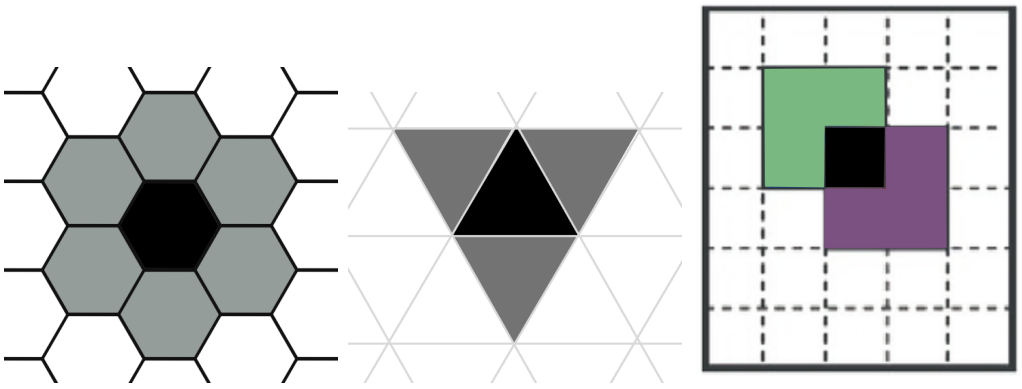
Por otro lado, GoL se compone de dos estados que denominó "vivas" y "muertas". De igual forma Stephen Wolfram utiliza dos estados en ECA.



(a) Vecindad de Neumann (b) Vecindad de Moore

(c) Vecindad radio uno

Figura 2.1: Las vecindades 2.1a y 2.1b son para CA bidimensionales, la vecindad 2.1c es para CA unidimensionales. La célula negra representa la célula actual y, junto con las grises, conforma su vecindad.



(a) Vecindad hexagonal (b) Vecindad triangular (c) Vecindad de Margolus

Figura 2.2: Vecindades para CA bidimensionales. En 2.2a y 2.2b, la célula negra es la célula actual y las grises conforman su vecindad. En 2.2c, la célula negra es la célula actual, mientras que las células verdes y moradas representan las vecindades activas en tiempos pares e impares, respectivamente.

t	$\vec{\sigma}(t)$										
0				$\ldots$	q	p	q	$\ldots$			
1			$\ldots$	q	p	p	p	q	$\ldots$		
2		$\ldots$	q	p	p	p	p	p	q	$\ldots$	
3	$\ldots$	q	p	p	p	p	p	p	p	q	$\ldots$

Tabla 2.1: Evolución temporal de un CA unidmensional con una configuración inicial finita. Donde p representa estados activos y q estados inactivos. Los conjuntos $\vec{\sigma}(t)$ y $\vec{\sigma}(t+1)$ son finitos.

Idealmente, el espacio celular $\mathcal{L}$ es infinito, lo que permitiría infinitas combinaciones de estados. Sin embargo, esto es inviable computacionalmente, por lo que, al igual que el CA de John von Neumann, se requiere establecer una restricción de finitud.

Definición 2. *Sea $\mathcal{R}$ la regla local de un CA. Un estado $\sigma_q \in \mathcal{S}$ es inactivo si $\mathcal{R}(\sigma_q, \sigma_q, \ldots, \sigma_q) = \sigma_q$.*

Es decir, si una célula y todos sus vecinos están en estado inactivo, la célula permanece inactiva tras aplicar $\mathcal{R}$. Cabe notar que todos los vecinos deben de estar en dicho estado, por lo que, si alguno cambia, podría reactivarse.

Definición 3. *Sea $\sigma_q \in \mathcal{S}$ un estado inactivo, $\sigma_p \neq \sigma_q \in \mathcal{S}$ un estado no inactivo y sea $\mathcal{L}$ un espacio celular infinito. Se considera que un CA tiene configuración finita si solo un número finito de celdas están en estados activos σ_p.*

Si la configuración inicial es finita, entonces, la evolución en el siguiente paso de tiempo, también sera finita (ver tabla 2.1).

Sea $\mathcal{C} = \mathcal{S}^{\mathcal{L}}$ el conjunto de todas las configuraciones. Definimos $\mathcal{C}_F \subset \mathcal{C}$ como el subconjunto de todas las configuraciones finitas. Esta restricción es necesaria para hacer computacionalmente viable la simulación de la evolución de un CA.

Finalmente, resulta necesario definir un estado estable el cual servirá mas adelante para establecer una clasificación.

Definición 4. *Dado un CA y un estado $s \in \mathcal{S}$, se dice que s es un estado estable si $\mathcal{R}(s, s, \ldots, s) = s$.*

Esto es, un estado estable s que evoluciona siempre en si mismo, por lo que $\sigma(t) = \sigma(t+1)$.

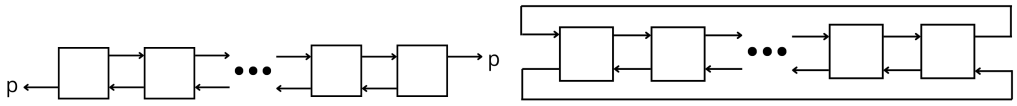
2.1.3. Condiciones de frontera

Idealmente, los CA pueden evolucionar en espacios infinitos; sin embargo, para su simulación computacional es necesario establecer límites. Estos limites, llamados condiciones de frontera, restringen el espacio celular $\mathcal{L}$.

Definición 5. *Un CA es finito si el espacio celular $\mathcal{L} \subseteq \mathbb{Z}^D$ es finito; en caso contrario, se denomina infinito.*

En un CA finito se consideran principalmente dos tipos de condiciones de frontera: abierta y periódica.

- **Condición de frontera abierta**: Se les asigna un estado fijo a todos aquellos vecinos faltantes en las células de la frontera.
- **Condición de frontera periódica**: Los vecinos faltantes de las células en la frontera son otras células en la frontera. La figura 2.3b muestra cómo se conectan las células de la frontera en un espacio unidimensional.



(a) Condición de frontera abierta (b) Condición de frontera periódica

Figura 2.3: Condiciones de frontera de un CA unidimensional con dos vecinos. La figura 2.3a asigna un estado no inactivo al vecino faltante. La figura 2.3b entabla relación de vecindad entre las células en la frontera formando un anillo.

Las condiciones de frontera periódica relacionan a los CA finitos y los CA infinitos con configuraciones periódicas.

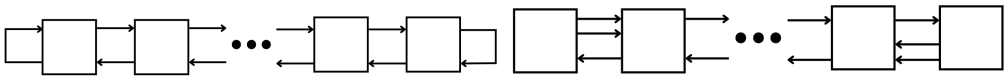
Definición 6. *Una configuración $\vec{\sigma} \in \mathcal{C}$ de un CA D-dimensional se dice periódica si es invariante ante un conjunto de traslaciones linealmente independientes en D direcciones. Es decir:*

$$\vec{\sigma} = T_i^{n_i}(\vec{\sigma}),\ \forall i \in \{1, 2, \ldots, D\}$$

donde $T_i^{n_i}$ denota un desplazamiento de n_i posiciones en la dimensión i.

Definimos entonces a $\mathcal{C}_P \subset \mathcal{C}$ como el conjunto de todas las configuraciones periódicas de un CA. Estas configuraciones preservan la condición de frontera periódica en cada traslación.

La frontera reflectiva asigna a los vecinos faltantes el mismo estado de la célula en la frontera (figura 2.4a). En este caso, las células de la frontera son vecinas de sí mismas. La frontera reflexiva, en cambio, imita el estado de los vecinos mas próximos de la célula en el borde (figura 2.4b), generando una doble relación de vecindad en los extremos del CA.



(a) Condición de frontera reflectiva (b) Condición de frontera reflexiva

Figura 2.4: Condiciones de frontera de un CA unidimensional con dos vecinos. La figura 2.4a muestra que las células en la frontera son vecinas de si misma, mientras que la figura 2.4b muestra que las células vecinas de las células de la frontera pueden ser múltiples vecinos.

2.1.4. Regla local

Sea $\mathcal{S}$ el conjunto de estados y $m = |\mathcal{N}|$ el numero de vecinos. La función de transición o regla local se define como

$$\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$$

donde $\mathcal{R}$ asigna un nuevo estado a cada célula en función de los estados de sus m vecinos (incluyéndose a sí misma si aplica).

Para CA unidimensionales, dado un rango arbitrario r, la regla de un CA unidimensional puede ser escrita como:

$$\sigma_i(t+1) = \mathcal{R}(\sigma_{i-r}(t), \ldots, \sigma_i(t), \ldots, \sigma_{i+r}(t)),\ \mathcal{R} : S^{2r+1} \to S$$

Donde el rango r especifica el tamaño de la vecindad para cada célula y cada una de ellas puede adquirir un valor k. Por esto, $\mathcal{R}$ puede ser definido especificando el valor asignado para cada una de las k^{2r+1} posibles $(2r+1)$-tuplas dada una vecindad.

Un ejemplo de regla local es GoL. En este caso, el conjunto de estados es binario $\mathcal{S} = \{0, 1\}$, donde 0 representa células muertas y 1 representa células vivas. La regla de transición se define como:

$$\mathcal{R} : \{0, 1\}^9 \to \{0, 1\}$$

ya que la vecindad de Moore en 2D involucra 8 vecinos mas la propia célula, es decir, $m = 9$. GoL sigue las siguientes tres reglas:

- **Nacimiento**: Una célula muerta en el tiempo t nace en el tiempo $t+1$ si tiene exactamente tres vecinos vivos en el tiempo t.
- **Muerte**: Una célula viva en el tiempo t muere en $t+1$ por:
  - sobrepoblación si hay más de tres vecinos vivos en t.
  - soledad si hay menos de tres vecinos vivos en t.
- **Supervivencia**: Una célula viva en el tiempo t se mantiene viva en $t+1$ si tiene exactamente tres vecinos vivos en t.

Se puede usar la notación *Birth/survival* la cual es un dupla de listas que almacenan el numero de vecinos vivos que tiene que haber en la vecindad para nacer y para sobrevivir respectivamente. La configuración para el juego de la vida "tradicional" es $B3/S23$, es decir, se requieren 3 células vivas para nacer y se requieren 2 o 3 células vivas para sobrevivir.

Una regla es semi-totalistica si la transición depende de la suma de los valores en la vecindad, pero distingue entre diferentes posiciones, como el centro y los bordes. GoL es un ejemplo de regla semi-totalistica porque cuenta el numero de vecinos vivos, pero trata de forma distinta el estado de la célula central.

En cambio, una regla es totalistica si la transición depende unicamente de la suma de los estados en todos los vecinos, sin distinguir posiciones. Por ejemplo, para un CA unidimensional con $|\mathcal{S}| = 3$ estados y radio $r = 1$, solo hay $(|\mathcal{S}| - 1)|\mathcal{N}| + 1 = (2-1) \cdot 3 + 1 = 4$ posibles valores de suma.

Ahora, considerando un $k = 2$ (es decir, $\sigma_i \in \{0, 1\}$), un rango r ($|\mathcal{N}| = 3$) y considerando la regla $\mathcal{R} : \{0,1\}^3 \to \{0,1\}$ que se define como:

$$\phi(\sigma_{i-1}(t), \sigma_i(t), \sigma_{i+1}(t))$$

La dinámica esta dada por las posibles configuraciones que las tres células adyacentes pueden adquirir en el tiempo t. Estos son el total de patrones que podemos encontrar con tres células. Cada que encontremos un patrón podemos asignarle un nuevo estado a $\sigma_i(t+1)$ ($\{0,1\}$). Esto se traduce a que podemos encontrar $k^{|\mathcal{N}|} = 2^3 = 8$ patrones posibles, y dado que cada patrón puede asignar uno de los k estados, el numero total de reglas es $k^{k^{|\mathcal{N}|}} = 2^8 = 256$, como se muestra en la tabla 2.2.

Esta reglas se pueden representar de forma tabular y también como un numero decimal. La tabla 2.3 muestra la asociación entre la forma tabular y la regla decimal.

111	110	101	100	011	010	001	000
↓	↓	↓	↓	↓	↓	↓	↓
$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$

Tabla 2.2: Cada posible combinación de tres células puede adquirir uno de dos valores. Cuando la célula evaluado encaja en este patrón su valor cambia al establecido. Este conjunto de 256 forman parte del ECA.

Binario	111	110	101	100	011	010	001	000	
Decimal	7	6	5	4	3	2	1	0	**regla**
Valor	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	1	1
					⋮				⋮
	0	0	0	1	1	1	1	0	30
					⋮				⋮
	0	1	1	0	1	1	1	0	110
					⋮				⋮
	1	1	1	1	1	1	1	1	255

Tabla 2.3: Los valores binarios representan las posibles configuraciones de estados que pueden aparecer en el espacio, cada uno asociado a un valor decimal y el valor es la potencia asociada a esa combinación. Para convertir la regla tabular en la decimal solo es necesario convertir el numero binario.

Nótese que la regla decimal solo aplica cuando las condiciones son las mismas, es decir, cuando el numero de vecinos $|\mathcal{N}|$ y estados $|\mathcal{S}|$ son el mismo, a cada regla se le puede asociar un numero entero único, pero dos reglas con vecinos y estados diferentes pueden compartir regla. Por ejemplo, considerando la regla 10 con $|\mathcal{N}| = 2$ y $|\mathcal{S}| = 2$ es 1010, la cual es completamente cuando $|\mathcal{N}| = 3 : 000000101$.

Definición 7. *Sea $\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$ la regla local de CA. La tupla $(s_1, s_2, \ldots, s_m) \in \mathcal{S}^m$ es un un termino mínimo de la regla (RMT, Rule Min Term) de la regla $\mathcal{R}$. Sea $\mathcal{S} = \{0, 1, \ldots, k-1\}$, entonces el RMT es $r = s_1 \cdots k^{m-1} + s_2 \cdots k^{m-2} + \cdots + s_{m-1} \cdot k + s_m$. La salida de la regla se denota como $\mathcal{R}(r)$.*

Cada transición de la regla esta dada por un RMT junto con su salida. Por ejemplo, en ECA ($k = 2$, $m = 3$), cada transición es $(x, y, z, \mathcal{R}(x, y, z))$. Esta transición se considera activa si $\mathcal{R}(x, y, z) \neq y$ e inactiva en el caso contrario.

Hasta ahora, hemos considerado CA uniformes, donde todas las células com-

parten la misma regla local $\mathcal{R}$. Sin embargo, también existen CA no uniformes o híbridos, en los que cada célula puede seguir una regla distinta, es decir, un vector de reglas.

Definición 8. *Un vector de reglas de un CA no uniforme sobre un espacio $\mathcal{L}$ es $\mathcal{R} = \langle \mathcal{R}_0, \mathcal{R}, \ldots, \mathcal{R}_n - 1 \rangle$ donde $\mathcal{R}_i$ es la regla usada por la célula $i \in \mathcal{L}$.*

Por ejemplo, si consideramos un CA no uniforme con 5 células cuyo vector de reglas es $\mathcal{R} = \langle 90, 150, 90, 5, 150 \rangle$ implica que el primer y el tercer elemento aplican la regla 90, el segundo y el quinto la regla 150 y el cuarto la regla 5.

2.1.5. Clasificación

Stephen Wolfram, al estudiar ECA, descubre que existen comportamientos compartidos por ciertas reglas y que dicho comportamiento es inherente de la configuración inicial del espacio celular. Es por esto que define las siguientes clases de comportamiento, ilustradas en la figura 2.5:

I CA que evolucionan a un estado uniforme.

II CA que evolucionan a estados cíclicos aislados.

III CA que evolucionan en comportamientos caóticos.

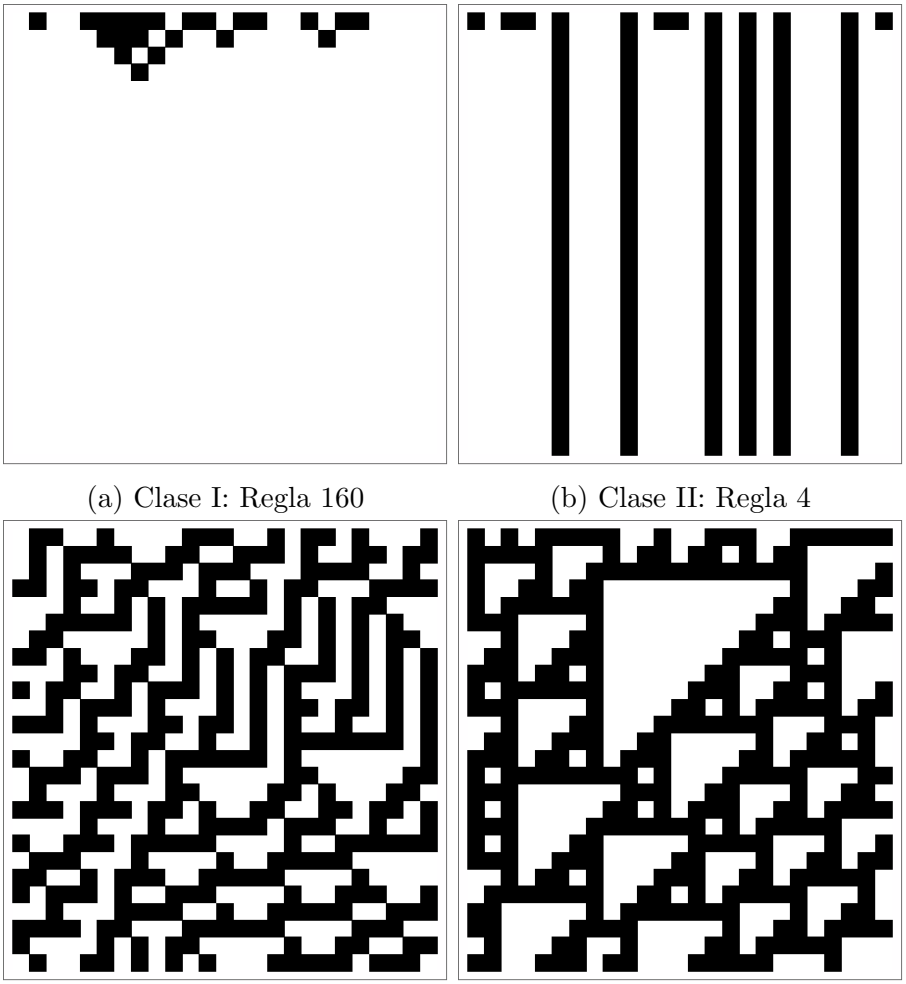
IV CA que evolucionan en estados complejos aislados.

Sin embargo, aunque esta clasificación funciona para ECA existen reglas bajo diferentes configuraciones que pueden pertenecer a múltiples clases o, en el peor de los casos, no pertenecen a ninguna de ellas [2].

Es por esto que en [3] proponen una nueva clasificación siguiendo ciertas limitaciones, entre las cuales están:

- Esta clasificación es para CA finitos.
- Cada clase es un subconjunto de la clase siguiente.

Definición 9. *Un CA es de clase I si existe un estado estable $s \in \mathcal{S}$ tal que todas las configuraciones finitas evolucionan a una configuración homogénea de s.*



(a) Clase I: Regla 160

(b) Clase II: Regla 4

(c) Clase III: Regla 30

(d) Clase IV: Regla 110

Figura 2.5: Clasificación de Wolfram. las figuras 2.5a, 2.5b, 2.5c y 2.5d evolucionan 25 pasos una configuración inicial aleatoria con frontera cíclica.

Es decir, dada cualquier configuración inicial en un CA finito, todas las células adquieren el estado s tras cierto numero de pasos evolutivos. Esto se aplica para todas las finitas configuraciones iniciales.

Definición 10. *Un CA es de clase II si existe un estado estable $s \in \mathcal{S}$ tal que se repite a n pasos evolutivos.*

Esto implica que la evolución se repite cada n numero de pasos ($\sigma(t) = \sigma(t+n)$). Resulta evidente que la clase I esta contenida en la case II haciendo que $n = 1$, lo que implica un periodo de uno.

Una clasificación es s-finita si solo difiere del estado estable en una región finita.

Definición 11. *Un CA es de clase III si existe un estado estable $s \in \mathcal{S}$ tal que para cualquier par de configuraciones s-finitas C_1 y C_2, es decidible si C_1 evoluciona a C_2.*

Es decir, si podemos determinar en tiempo finito si el CA evoluciona de C_1 a C_2, entonces pertenece a clase III. En caso contrario, pertenece a clase IV, la cual contiene a todos los CA [3].

2.2. Redes neuronales

Las redes neuronales artificiales (ANN, *Artificial Neural Network*) es un modelo inspirado del pensamiento humano, mas específicamente, en como un conjunto de neuronas interconectadas comparten información.

Las ANN son una conjunto de neuronas artificiales interconectadas que realizan cálculos y transmiten información para realizar una operación. Su inspiración es el cerebro.

2.2.1. Neurona biológica

Las neuronas son células especializadas del sistema nervioso cuyo objetivo es recibir, procesar y transmitir información por medio de señales químicas y eléctricas.

Cada neurona se compone (principalmente) de tres elementos como se muestra en la figura 2.6:

- Las dendritas que reciben señales provenientes de otras neuronas.
- El núcleo o cuerpo, que integra las señales proveniente de las dendritas y produce una nueva señal eléctrica.
- El axón, el cual conduce la señal producida por el cuerpo hacia otras neuronas haciendo sinapsis, es decir, transmite la señal de una célula a otra.

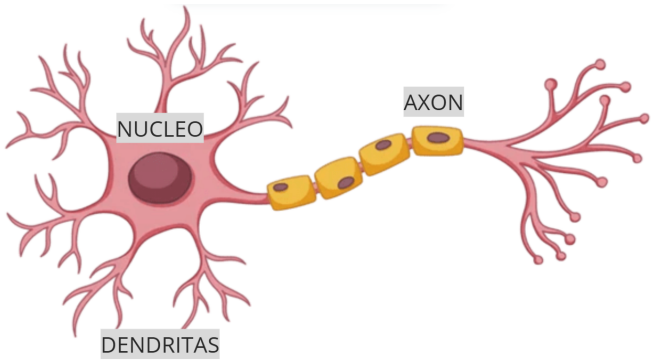


Figura 2.6: Representación de una neurona [4]. Se encuentra señaladas las dendritas, el núcleo y el axón, encargado de recibir, procesar y transmitir la información respectivamente.

Las redes neuronales naturales son un sistema de neuronas que permite el procesamiento cerebral de la información, son altamente complejas, no lineales y con procesamiento en paralelo.

2.2.2. Neurona artificial

La neurona artificial se considera como una unidad de procesamiento de información. Sigue la misma estructura del modelo biológico con algunas consideraciones adicionales:

- Las dendritas equivalen al conjunto de datos de entrada. Sin embargo, no toda la información tiene la misma relevancia, para esto se le asigna a cada entrada un peso sinaptico.
- El cuerpo de se sustituye por dos funciones: (1) Una función de agregación que combina todos los datos de entrada (generalmente es una combinación lineal entre las entradas y los pesos sinapticos) y (2) una función de activación que determina si la neurona esta activa y en que grado.

Definición 12. *Una neurona artificial es una función* $f : \mathbb{R}^n \to \mathbb{R}$ *dada por:*

$$f(x) = \phi(\omega(X, W, b))$$

donde:

- $X = \{x_1, x_2, \ldots, x_n\} \in \mathbb{R}$ *es el conjunto de datos que recibe cada neurona.*
- $W = \{w_1, w_2, \ldots, w_n\} \in \mathbb{R}$ *es el conjunto de pesos sinapticos que indican la importancia de cada entrada.*
- b *o* $w_0 \in \mathbb{R}$ *es un sesgo que desplaza la función de activación y controla el umbral de aceptación.*
- $\omega(X, W, b)$ *es una función de agregación* $\omega : \mathbb{R}^n \to \mathbb{R}$ *que combina las entradas. Usualmente se define como una combinación lineal:*

$$\omega(X, W, b) = \sum_{i=1}^{n} w_i x_i + b$$

- $\phi(\omega(X, W, b))$ *es la función de activación* $\phi : \mathbb{R} \to \mathbb{R}$ *que transforma* $\omega(X, W, b)$ *en una salida activada.*

La cual se suele representar con la figura 2.7.

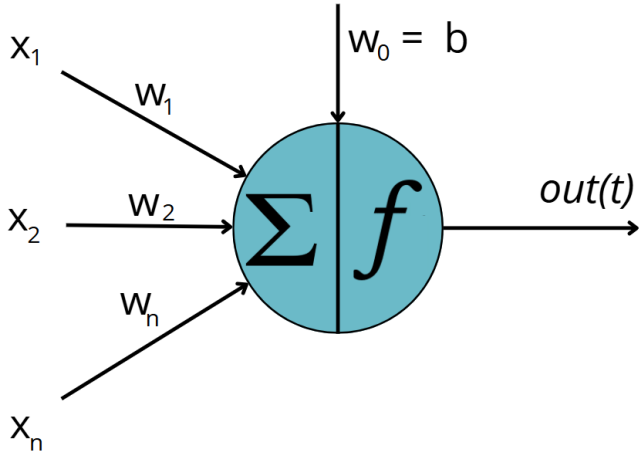


Figura 2.7: Representación grafica de una neurona artificial. Contiene las entradas $(x_1, x_2, \ldots, x_n)$, los pesos $(w_1, w_2, \ldots, w_n)$, el sesgo (w_0 o b), función de agregación (Σ), función de activación (f) y su salida.

2.2.2.1. Función de activación

La función de activación es la que determina el valor de salida de cada neurona. Este valor, normalmente, se asigna entre un umbral para que limiten el comportamiento de la red neuronal. A continuación se muestran algunas de las funciones de activación mas comunes con su respectiva grafica.

- **Lineal**: Utilizada para regresión lineal, la salida equivale a la función de agregación pero limita la capacidad de aprender patrones complejos.

$$\phi(z) = z$$

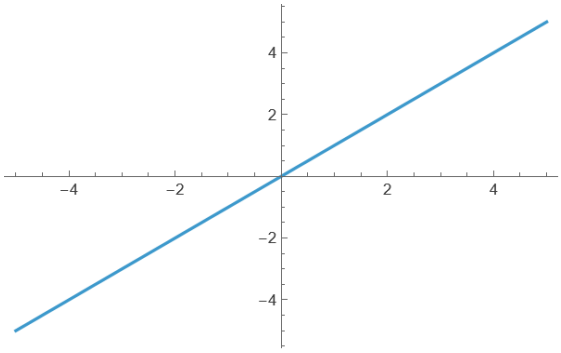


Figura 2.8: Función de activación lineal.

- **Escalón (binaria)**: Utilizada cuando se conoce el umbral de activación y usada para clasificación binaria.

$$\phi(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$$

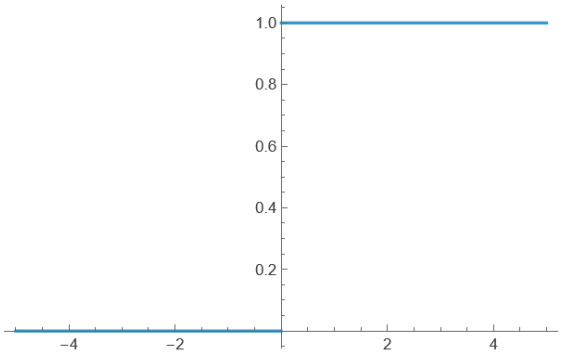


Figura 2.9: Función de activación escalón.

- **Sigmoide (Logística)**: La salida esta en un rango entre $[0, 1]$, lo que permite tratarlo como una probabilidad.

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

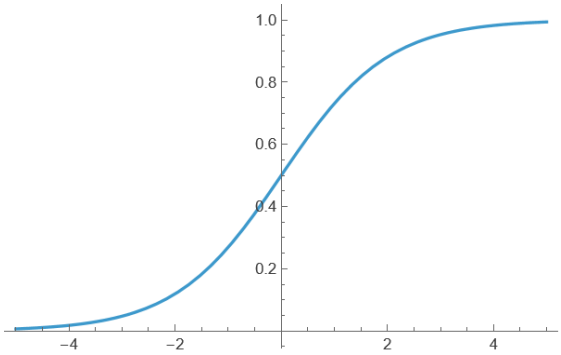


Figura 2.10: Función de activación sigmoide.

- **Tangente hiperbólica**: proporciona salidas en el rango $[-1, 1]$.

$$\phi(z) = \tanh(z) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

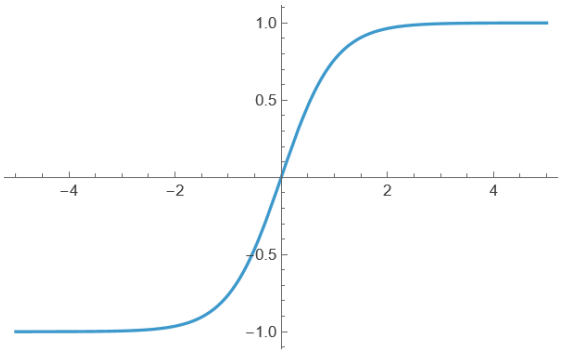


Figura 2.11: Función de activación tangente hiperbólica.

- **ReLU (*Rectified Lineal Unit*)**: Función recientemente utilizada por su rápida convergencia en el entrenamiento.

$$\phi(z) = \text{máx}(0, z)$$

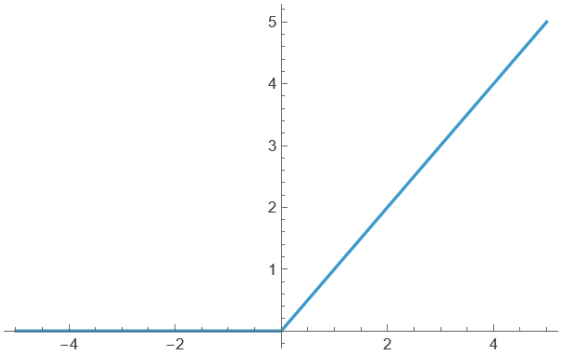


Figura 2.12: Función de activación ReLU.

2.2.3. Redes neuronales

Una neurona unicamente puede realizar una operación, para aumentar la capacidad de entender patrones mas complejos es necesario añadir múltiples neuronas interconectadas, lo que se conoce como red neuronal artificial.

Las neuronas interconectadas permiten transmitir y propagar información, tradicionalmente se estructuran en un modelo multicapa (ver figura 2.13) la cual incluye:

- Capa de entrada: Es la primera capa de procesamiento cuya principal tarea es recibir los datos.
- Capa o capas ocultas: Es un conjunto de múltiples capas y son el componente con mayor relevancia a la hora de procesamiento.
- Capa de salida: Esta ultima capa se encarga de darle formato a los resultados para mostrarlos.

2.2.4. Aprendizaje en redes neuronales

Con las condiciones ideales, se dan los pesos de cada neurona y el sistema es capaz de hacer su trabajo sin problemas pero es necesario de un "entrenamiento" que permita a la ANN modificar los pesos de cada neurona. A continuación se define la función de perdida, optimización y entrenamiento que permite definir los pesos sinapticos de cada neurona, entrenar al sistema para que realize su tarea especifica y medir el nivel de error.

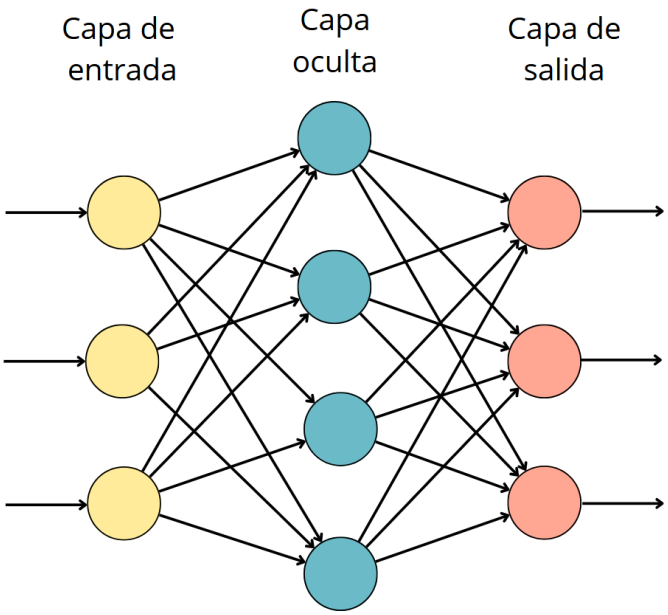


Figura 2.13: Representación del perceptrón multicapa totalmente conectado no recurrente con una sola capa oculta.

2.2.4.1. Función de perdida

La función de perdida se usa para medir el rendimiento del modelo. Calcula la desviación de la predicción realizada por el modelo respecto a la clase que corresponde. Optimizar un modelo es entonces ajustar los parámetros para disminuir la función de perdida, es decir, que entre mejor sea la tasa de predicciones correctas menor es la función de perdida.

Es una medida que nos permite conocer la exactitud del modelo y que tan lejos se encuentra de la solución optima.

La función de perdida no representa unicamente una distancia escalar entre valor obtenido y el valor esperado, sino que puede funcionar como parámetro para una función de optimización, por ejemplo gradiente descendiente utiliza el gradiente de la función de perdida", es decir, la derivada de una función con múltiples variables.

Es importante que la función de perdida sea diferenciable. Usando el gradiente de la función de perdida es posible conocer la dirección hacia la cual debe moverse para disminuir la perdida.

Las funciones de perdida se pueden utilizar para regresión y para clasificación.

La primera mide el error en valores continuos, usado para estimar valores contables como el precio, tamaño, tiempo. La segunda mide el errores en valores discretos, como la clasificación o la categoría.

Diferentes funciones de perdida priorizan diferentes tipos de error, la elección de la función de perdida dependerá de la tarea que se desea realizar. A continuación se muestran algunas funciones de perdida.

Regresión Sirve para problemas de regresión lineal y regresión polinómica. Su salida son valores continuos que determinan la relación entre una o varias variables independientes. La función no solo determina si la salida es incorrecta, sino que ademas devuelve el grado en que diverge del resultado objetivo.

A continuación se listan algunas funciones de perdidas comúnmente utilizadas donde n el numero de pruebas realizadas, y_i es el valor y $\hat{y}_i$ es el valor obtenido:

- **MSE (*Mean Squared Error*)**. El error cuadrático medio o también conocido como "L2 loss" la cual se define como:

$$MSNE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

  Elevar al error al cuadrado indica que siempre sera positivo, por lo tanto, esta función solo devuelve la magnitud y no la dirección. Elevar al cuadrado también asigna mayor prioridad a los errores mas grandes.Esta función es útil cuando el resultado esperado se asume que tiene una distribución Gaussiana.

- **MSLE *Mean Squared Logarithmic Error***:El error logarítmico cuadrado medio se usa cuando existe un amplio rango de valores. Se define como:

$$MSLE = \frac{1}{n}\sum_{i=1}^{n}(\log(1+\hat{y}_i) - \log(1+y_i))^2$$

  Grandes penalizaciones a problemas grandes puede ser contraproducente, MSLE evita esto.

-

Clasificación

2.2.4.2. Optimización

2.2.4.3. Entrenamiento

2.2.5. Redes neuronales

2.2.6. Arquitecturas

Capítulo 3

Estado del Arte

3.1. Introducción

En 1948, John von Neumann, con ayuda de Stanislaw Ulam, propuso un Autómata Celular (CA, *Cellular Automaton*) como un modelo matemático con la capacidad de autoreproducción universal, el cual desarrollo en su libro *Theory of Self-Reproducing Automata* [1]. Hoy en día representa un modelo fundamental para el estudio de sistemas complejos, computación distribuida y procesos autoorganizados.

Diversos autores, como Stephen Wolfram [5] y Erick W. Weisstein [6] coinciden en caracterizar a los CA como sistemas discretos compuestos por células dispuestos sobre una retícula[1] (*lattice*). Cada celda puede adoptar un valor de un conjunto finito de estados, y su actualización ocurre en pasos discretos de tiempo, determinada por una función de transición que depende únicamente de su vecindad local.

Aunque existen variaciones en la terminología (celda, célula, sitio, color), la estructura formal es ampliamente aceptada y utilizada en campos como la física, biología computacional, procesamiento de imágenes, teoría de la computación, entre otros. Más recientemente, se han integrado técnicas de aprendizaje supervisado, en particular Redes Neuronales Artificiales (ANN), dando lugar a los Autómatas Celulares Neuronales (NCA, *Neural Cellular Automata*).

[1] Conjunto de líneas o elementos de estructura lineal que se cruzan o entrelazan estrechamente [7]

El presente estado del arte revisa estas propuestas híbridas, enfatizando sus aplicaciones en morfogénesis artificial, clasificación de patrones, generación de texturas dinámicas y modelado en mallas tridimensionales. Además, se identifican retos en términos de escalabilidad, generalización y control de dinámicas emergentes, aspectos que motivan el desarrollo de nuevas variantes de NCA.

3.2. Simulación mediante autómatas celulares

A lo largo del tiempo, se han propuesto múltiples variantes del modelo clásico de autómatas celulares, adaptadas a contextos no estructurados, probabilísticos, heterogéneos o continuos. A continuación, se describen algunas de estas extensiones para el modelado de sistemas complejos.

- Los autómatas celulares en grafos (GCA, *Graph Cellular Automata*) generalizan la estructura reticulada clásica permitiendo vecindades no uniformes: cada célula puede tener un número arbitrario y variable de vecinos. Esta flexibilidad los hace adecuados para modelar fenómenos sobre grafos, como redes sociales o flujos en infraestructura. Sin embargo, diseñar manualmente las reglas de transición en estos entornos es una tarea compleja, lo que ha motivado propuestas como los Graph Neural Cellular Automata (GNCA), que permiten aprender dichas reglas automáticamente [8].
- Los autómatas celulares probabilísticos (PCA, *Probabilistic Cellular Automata*) extienden el modelo clásico al incorporar reglas de transición estocásticas. En lugar de una evolución determinista, los estados futuros dependen de distribuciones de probabilidad condicionadas por la vecindad, lo que permite capturar incertidumbre, ruido y comportamientos emergentes. Son especialmente útiles para simular dinámicas como propagación de epidemias o sistemas sociales, donde los mecanismos de transición no siempre están completamente definidos [9].
- Los autómatas celulares no homogéneos (INCA, *Inhomogeneous Cellular Automata*) permiten que diferentes células tengan diferentes funciones de transición. Para esto, se utilizan dos planos llamados *data plane* el cual contiene la evolución del sistema; y *control plane* el cual define las reglas de transición para cada célula. Esto permite estudiar el comportamiento y la interacción entre diferentes reglas que comparten espacio [10].
- Los autómatas celulares continuos (CCA, *Continuous Cellular Automata*) pueden ser continuos en los estados, en el tiempo y en el espacio [11]:

- Los estados continuos adquieren valores reales y no solo enteros.
- El tiempo continuo se define mediante una función que determina el intervalo de espera para actualizar la célula en función de su estado actual.
- El espacio continuo se obtiene al asignar diferentes pesos a las células dentro de la vecindad [12].

Si bien estas variantes han expandido el alcance de los CA, un desafío recurrente es la correcta definición de las reglas de transición para reproducir un fenómeno deseado. Este reto ha motivado la introducción de los Autómatas Celulares Neuronales (NCA, *Neural Cellular Automata*), en los cuales la función de transición no se especifica explícitamente, sino que es aprendida mediante una Red Neuronal Artificial (ANN, *Artificial Neural Network*). De este modo, las reglas locales emergen directamente de los datos, lo que facilita modelar dinámicas de alta complejidad.

Un primer avance en esta línea fue presentado por Gilpin [13], demostrando que una Red Neuronal Convolucional (CNN, *Convolutional Neural Network*) puede aprender el comportamiento de un CA clásico. Su estudio destaca que tanto los CA como las CNN comparten dos propiedades fundamentales: (1) la actualización basada en una vecindad o *kernel*, y (2) la aplicación simultánea de dicha función en todo el espacio.

En particular, Gilpin entreno una CNN con un *kernel* de 3×3 (mismo tamaño que la vecindad de Moore) para imitar el comportamiento del *glider*[2] en el Juego de la Vida (GoL, *Game of Life*) de John Conway. Este trabajo estableció una conexión entre los CA y las CNN, mostrando que las ANN puede actuar como mecanismos de transición local aprendidos.

Posteriormente, esta línea de investigación fue extendida por Mordvintsev et al., quienes exploraron representaciones continuas para morfogénesis artificial (*Growing NCA*), y por Niklasson et al., que aplicaron el enfoque a la generación de texturas dinámicas sobre superficies tridimensionales (*Mesh NCA*). A su vez, otros autores han propuesto aplicaciones en robótica, clasificación de patrones y simulación de sistemas fotónicos, consolidando a los NCA como una herramienta para el modelado de sistemas autoorganizados.

[2]Una de las configuraciones mas importantes por su capacidad de moverse en el espacio

3.3. Morfogénesis

Mordvintsev et al. en [14] diseñaron un NCA que permite a una célula crecer para formar un patrón, persistir en el tiempo, regenerarse e incluso rotar. Para esto diseñaron un modelo que depende de un CA continuo con un vector de estados de 16 dimensiones integrando gradientes espaciales mediante filtros de Sobel.



Figura 3.1: Evolución del *emoji* de una lagartija a partir de una única célula activa después de 219 pasos evolutivos [14].

El NCA diseñado tiene las siguientes características (las cuales fueron retomadas por múltiples autores que tomaron a [14] como referencia):

- El vector de cada célula (cada valor en un rango de $[0, 1]$) incluye los canales RGB y un canal α que funciona como umbral para definir si una célula está viva o muerta (establece el resto de valores del vector a 0 si $\alpha \leq 0.1$).
- Considerando las dimensiones (largo, ancho, 16) y un kernel de 3×3 se calcula el gradiente en ambas direcciones (x y y) con un filtro Sobel para producir el nuevo vector que alimentará a la red neuronal:

$$16(\text{estado propio}) + 16(\text{gradiente en } x) + 16(\text{gradiente en } y) = 48$$

- Se usaron convoluciones de 1×1 con una función $ReLU$, alrededor de 8000 parámetros entrenables y devuelve un δ que se suma al estado actual de célula antes del siguiente paso.
- Se omite el cálculo para las células muertas y la actualización de cada célula se realiza con una probabilidad del 50 %. La actualización debe ser estocástica para lograr un mejor rendimiento [15].

Eyvind Niklasson et al. [15] compararon el rendimiento del modelo en [14] con actualización síncrona y asíncrona. Donde, tras haber realizado pruebas con diferentes tamaños de paso, concluyeron que la función de pérdida crece exponencialmente cuando la actualización es síncrona. Sin embargo, esto ocurre únicamente cuando el tamaño de paso es mayor a uno.

Una hipótesis de este comportamiento es que al tener células que se actualizan en diferentes periodos de tiempo, cada célula puede conocer parte del contexto de su vecindad. Esto no se logra únicamente conociendo los valores actuales de cada vecino.

Se realizaron los siguientes experimentos en [14]:

1. Crecer una única célula activa para formar una figura dada mediante *back-propagation*. Cuando él CA alcanza la figura objetivo, al seguir aplicando las reglas de evolución, puede crecer de manera infinita, persistir en la figura, matar a todas las células o generar patrones caóticos o repetitivos.
2. Convertir las figuras objetivo en atractores usando *sample pool*. Se entrenó mediante múltiples condiciones iniciales, obtenidas a partir de entrenamientos fallidos, para llegar al objetivo desde diferentes configuraciones.
3. Regenerar partes de la figura. Se eliminaron partes de la figura objetivo y se añadieron al conjunto de entrenamiento.
4. Rotación de la figura objetivo. Dado que los filtros de Sobel funcionan como direcciones ortogonales, se puede aplicar una rotación mediante:

$$\begin{bmatrix} K_x & K_y \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \cdot \begin{bmatrix} Sobel_x & Sobel_y \end{bmatrix}$$

   para modificar el ángulo sin necesidad de reentrenar el modelo.

3.3.1. NCA Isotrópicos

En [16] [17] [18], Mordvintsev et al. proponen una nueva configuración que no tenga una dirección preferida, la cual nombraron Autómatas Celulares Neuronales Isotrópicos (IsoNCA, *Isotropic Neural Cellular Automata*).

Para volver esta regla isotrópica es necesario que no dependa de la dirección (no distinguir "arriba" de "izquierda"). Esto permite crear rotaciones o simetrías sin que el sistema se descontrole.

Se sustituyen las funciones de los filtros de Sobel en [14] por un único filtro Laplaciano:

$$K_{lap} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & -12 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

el cual se concatena al vector de cada célula y alimenta a una ANN de 9408 parámetros.

A continuación, se describen los resultados obtenidos con dos diferentes enfoques:

1. Estrategia de *seed* (semilla) estructurada: Es necesario de (al menos) tres células con valores diferentes no colineales para definir la orientación y la posición. Modificar la posición de estos puntos implica una evolución acorde a la posición de los mismos.

2. Estrategia de una única *seed*: Se modifica la función de pérdida para que sea invariante ante rotaciones y reflejos. Para evitar comparar la figura resultante con todas las rotaciones posibles, se mapean ambas figuras a coordenadas polares y, para cada posible ángulo de rotación, se calcula la diferencia. Para disminuir la complejidad computacional, se aplica la transformada rápida de Fourier y haciendo uso del teorema de la convolución:

   $$s * t = \mathcal{F}^{-1}(\mathcal{F}(S) \cdot \mathcal{F}(T))$$

   se obtienen todas las diferencias únicamente como una multiplicación de transformadas.

   Para evitar que se creen patrones simétricos mezclados (Figuras con cabeza-cabeza o cola-cola) se añade un nuevo canal llamado *binary auxiliar channel* que asigna valores de -0.5 y 0.5 a la parte superior e inferior de la imagen.

   Con estas nuevas configuraciones se puede obtener el mismo resultado que en [14] pero sin la dependencia de una dirección explícita y con una ANN con menos parámetros.

3.3.2. Autoclasificación

Mordvintsev et al. en [19] utilizan los NCA para la autoclasificación de MNIST[3] las cuales fueron binarizadas con un umbral de 0.1.

El objetivo es que, dada una configuración inicial de células vivas, el sistema sea capaz de identificar la forma únicamente mediante reglas locales, en este caso, que pueda identificar los números de MNIST.

Estos son los cambios de *MNIST CA* respecto al modelo original [14]:

[3]Conjunto de datos de imágenes 28×28 de números en el rango $[0, 9]$

- Vector de la célula: Originalmente se usaban los canales RGB, ahora se usan 10 canales que funcionan como distribución de probabilidad de cada posible número. Durante la evaluación se utiliza el mayor de ellos.
- Célula viva o muerta: en [14] se define mediante el canal α, pero las células muertas siguen siendo visibles para el resto. En [19] únicamente interactúan las células vivas.
- Topología: Todas las células son una única componente conexa.

Con el nuevo modelo, Mordvintsev et al. pudieron realizar clasificaciones con una precisión del 95.3 % con una función de pérdida L_2 y ruido añadido. Este experimento demostró que los NCA no solo pueden generar patrones y autorregenerarse, sino también realizar tareas de clasificación distribuidas sin una capa de decisión global.

Aunque los modelos *Growing CA* y *MNIST CA* son robustos frente a diferentes tipos de perturbaciones, su comportamiento se puede ver afectado por: estados individuales de las células, parámetros del modelo y el campo perceptivo (la manera en como las células perciben su vecindad).

Es por esto que Randazzo, et al. [20] buscan enfrentar estrategias adversarias en el mismo espacio celular para estudiar la reprogramación de los CA mediante dos experimentos:

1. Inyección de adversarios: Se entrena un nuevo modelo de CA, que se coloca en el entorno que ejecuta uno de los modelos originales, para secuestrar el comportamiento del colectivo.
2. Perturbación de estados: A un modelo ya entrenado, se le aplican perturbaciones a todas las células del sistema de forma simultánea.

Para la inyección de adversarios se utilizan dos modelos: el modelo original *MNIST CA* con probabilidad de 90 % y un nuevo modelo que clasifica a todos los números como “ocho” (10 %). Se “inyectan” múltiples células invasoras en figuras ya clasificadas las cuales muestran un comportamiento dominante: clasificando a todos los elementos como “ocho”. Además, una vez retiradas las células invasoras, el sistema es capaz de recuperarse la mayoría de las veces.

Sin embargo, al aplicar el mismo procedimiento para *Growing CA* los resultados no tuvieron el mismo rendimiento. Probaron modificar la morfología de la imagen

de una lagartija sin cola y con otro color, pero ninguna estrategia fue capaz de invadir por completo el modelo original.

Una hipótesis es que *MNIST CA* no necesita reconfigurar todo su valor para generar algo distinto a lo que ya generaba antes, pues el estado de la mutación es un estado ya considerado en el modelo original. Por otro lado, al cambiar el color y la forma en *Growing CA* se están explorando configuraciones nunca antes vistas por el modelo original.

Para el segundo experimento (perturbación de estados), se sigue la línea de modificar el color y la forma de la imagen de una lagartija. Se aplica una perturbación global de estados cada paso de tiempo con el objetivo de identificar la perturbación específica que resulte en un patrón estable.

En cada paso evolutivo se multiplica el vector de cada célula por una matriz simétrica A de 16×16 [14]. Para definir los valores de la matriz se inicia con una matriz simétrica I y, a diferencia de *Growing CA*, la configuración inicial se establece como la figura objetivo del modelo original.

Esta configuración permite crecer un sistema con perturbaciones, alterar su forma y color una vez crecido y modificar el impacto y el número de perturbaciones posibles. Al manejar matrices simétricas se puede representar como

$$A = Q\Lambda Q^T$$

donde:

- Λ es la matriz diagonal de los valores propios.
- Q es la matriz unitaria de los vectores propios.

Si $\Lambda = I \rightarrow A = I$ por lo que no existe perturbación (multiplicación por matriz identidad). Bajo esta premisa, se propone la siguiente fórmula:

$$A_k = Q(kD + I)Q^T$$

donde:

- A es la matriz original.
- k es el factor de escala.
- D es la dirección de la perturbación $\Lambda - I$ en el espacio de valores propios.

Si $k = 0$ no existe perturbación en el sistema:

$$A_k = Q(0D + I)Q^T$$
$$A_k = QIQ^T$$

Si $k = 1$ se conserva la perturbación original:

$$A_k = Q(1D + I)Q^T$$
$$A_k = Q(\Lambda - I + I)Q^T$$
$$A_k = Q\Lambda Q^T$$

Mostrando que se puede escalar el tamaño de la perturbación, incluso con perturbaciones inversas con $k = -1$.

Si consideramos dos perturbaciones cuyos vectores propios son similares: $A_k = Q(k_A D_A + I)Q^T$ y $B_k = Q(k_B D_B + I)Q^T$ podemos combinarlos como:

$$comb(A_k, B_k) = Q(k_A D_A + k_B D_B + I)Q^T = k_A A + k_B B + (1 - k_A - k_B)I$$

Esta nueva combinación permite configurar múltiples perturbaciones, las cuales se vuelven más sensibles conforme aumentan los k.

Estos experimentos muestran como los NCA pueden ser vulnerables a diferentes estrategias y como las mutaciones "exactas" pueden modificar la estructura de todo el sistema, mostrando un comportamiento similar a lo que ocurre en teoría de juegos evolutivos.

Todas estas representaciones han tenido un lugar en un espacio bidimensional (2D). A continuación se presentan algunas propuestas en espacios tridimensionales (3D).

3.3.3. Representaciones tridimensionales

Zhang et al. [21] proponen un modelo generativo tridimensional denominado *Generative Cellular Automata* (GCA), que extiende el paradigma de los CA hacia la generación progresiva de formas 3D. A diferencia de enfoques que intentan reconstruir directamente la forma completa, GCA formula el proceso como una cadena de Markov, donde cada paso corresponde a una transición local sobre una rejilla tridimensional.

El modelo se entrena para que estas reglas locales, al repetirse, converjan hacia formas específicas. Para ello, se representa la forma como puntos de superficie en

una rejilla 3D y se emplea una Red Convolucional Dispersa (*Sparse CNN*) que opera únicamente sobre las regiones activas.

Durante el entrenamiento, se utilizan secuencias ligeramente distintas a las cadenas de muestreo reales, pero que convergen hacia las formas del conjunto de datos. Esto permite que el modelo aprenda reglas de transición robustas, capaces de generar formas diversas desde configuraciones parciales. A través de una sola inferencia, el sistema puede muestrear instancias variadas de la distribución aprendida, logrando resultados competitivos en tareas de completado y generación de formas 3D.

Este enfoque representa una contribución relevante al campo de la morfogénesis computacional, al demostrar que las reglas locales autoorganizadas pueden escalar hacia dominios 3D complejos mediante arquitecturas neuronales adaptadas.

En [21] demuestran que la autoorganización local escala a generación 3D, por otro lado, Shyam Sudhakaran et al. [22] expanden el trabajo de [14] a tres dimensiones utilizando como espacio celular el juego de Minecraft[4]. El NCA entrenado puede construir estructuras de hasta 3584 bloques con 50 bloques únicos conservando propiedades regenerativas de su contraparte en 2D.

Una entidad en Minecraft se representa como una red 3D donde cada elemento es un vector que almacena (1) el tipo de bloque (incrustado en un vector *one-hot*), (2) su estatus (replicando el comportamiento del canal α de [14]) y (3) estados ocultos.

Se utilizan convoluciones para permitir la generación y manipulación de estructuras 3D. Además, se sustituye el uso de filtros de Sobel por una red de percepción aprendible con un kernel de $3 \times 3 \times 3$, *stride*[5] de 1 y canales de salida equivalentes a tres veces el número de canales del estado celular. Esta red permite que el sistema aprenda activamente a percibir su entorno local, adaptando sus filtros durante el entrenamiento y capturando patrones espaciales relevantes de manera dinámica.

Se utiliza una actualización estocástica y un umbral de activación similar a [14]. Finalmente, todos los modelos se inicializan con una distribución normal estándar, con media cero y desviación estándar en el rango de 0.0001 a 0.1, lo que proporciona un punto de partida estable para el entrenamiento y la evolución del sistema. Este sistema hereda las propiedades regenerativas de su versión 2D.

Finalmente, Horibe et al. [23] lleva las propiedades regenerativas al mundo físi-

[4]Permite crear estructuras estáticas (como árboles) y estructuras dinámicas (como máquinas voladoras)

[5]Cuántas posiciones se desplaza el *kernel* sobre la entrada en cada operación de convolución

co. Se implementa el enfoque regenerativo en robots blandos mediante NCA. Cada robot está definido por un arreglo de volúmenes de reposo para cada vóxel[6], que se activan mediante una señal global sinusoidal, provocando expansión y contracción volumétrica. Esta dinámica permite simular locomoción a través de ciclos de actuación, evaluando el desplazamiento del centro de masa como métrica de fitness. Los robots blandos se componen de distintos tipos de vóxeles: músculos en fase y contra-fase, huesos rígidos y flexibles, cada uno con un rol en la locomoción.

El modelo entrenado demuestra que los robots blandos pueden recuperar parcialmente su morfología original tras sufrir daño, y restaurar su capacidad de movimiento sin intervención externa. Este comportamiento emerge de la dinámica autoorganizada del NCA, que codifica implícitamente la forma objetivo y su capacidad de reparación.

Para evaluar la capacidad de locomoción y regeneración de los robots blandos generados mediante NCA, se emplea el simulador físico Voxelyze. Los experimentos se realizan en configuraciones 2D y 3D, utilizando rejillas de 7×7 y $9 \times 9 \times 9$ vóxeles respectivamente, con condiciones de frontera tipo Dirichlet que limitan el crecimiento morfológico a regiones internas (5×5 en 2D y $7 \times 7 \times 7$ en 3D).

Este entorno permite validar la capacidad autoorganizada de los NCA para generar morfologías funcionales y resilientes. Ademas, proporciona una base cuantitativa para comparar la eficiencia locomotora antes y después de procesos de regeneración. La estructura voxelizada facilita la integración de reglas locales y codificación distribuida, alineándose con marcos bioinspirados de morfogénesis computacional.

En conjunto, estos trabajos evidencian cómo los NCA pueden extender sus principios regenerativos y de autoorganización a dominios tridimensionales cada vez más complejos. Mientras GCA establece un marco generativo para formas 3D a partir de reglas locales aprendidas, los experimentos en Minecraft muestran que dichas propiedades pueden preservarse incluso en entornos con alta variabilidad estructural. Finalmente, la aplicación en robots blandos confirma el potencial bioinspirado de los NCA: no solo permiten generar morfologías funcionales, sino que además dotan a los sistemas físicos de resiliencia ante daños, abriendo camino hacia formas de robótica autoreparable.

Otra forma de implementar los NCA en 3D es mediante planos y texturas autoorganizadas que cubren la superficie de un objeto tridimensional.

[6]Unidad básica de un espacio tridimensional, similar a un píxel en imágenes 2D, pero con volumen

3.4. Mesh Neural Cellular Automata

Niklasson et al. [24] y Mordvintsev et al. [25] [26] introducen el concepto de *Self-Organising Textures*, cuyo objetivo es reproducir la apariencia global de una textura de referencia sin generar copias exactas. Para ello, entrenan un NCA con la misma arquitectura neuronal propuesta en [14], de modo que pueda capturar y generar patrones característicos como tableros de ajedrez, líneas o mosaicos. Aunque este enfoque tiene un carácter demostrativo, constituye una evidencia importante de que los NCA pueden aprender y generalizar comportamientos visuales locales en lugar de memorizar imágenes específicas.

Sobre esta base, Niklasson et al. proponen *Mesh Neural Cellular Automata* (MeshNCA) [27], un método diseñado para generar texturas dinámicas en superficies tridimensionales. A diferencia de los experimentos en cuadrículas bidimensionales, MeshNCA extiende el mecanismo de autoorganización a mallas arbitrarias, incluso cuando estas se encuentran en movimiento. El modelo mantiene coherencia espacial en distintos escenarios, muestra robustez frente a perturbaciones externas y permite controlar propiedades como densidad, dirección y rotación de los patrones emergentes. Además, varios MeshNCA pueden injertarse de manera conjunta, produciendo transiciones suaves entre texturas sin discontinuidades.

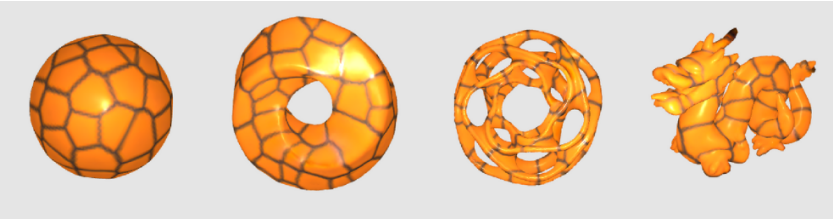


Figura 3.2: El entrenamiento se realiza sobre un poliedro simétrico y el NCA puede ser extrapolado a cualquier objeto tridimensional [14].

Sin embargo, este enfoque presenta un reto de escalabilidad. El número de células involucradas suele encontrarse en el rango de $10^4 \sim 10^5$, lo que limita la resolución alcanzable a entre 64×64 y 256×256 píxeles. Como el tiempo de entrenamiento escala de forma cuadrática con respecto al número de celdas, el aumento de resolución conlleva un cuello de botella en la propagación de información a lo largo de la malla.

Para superar esta limitación, Niklasson et al. proponen en [28] un modelo híbrido que combina NCA con Representaciones Neuronales Implícitas (INR, *Implicit Neural Representations*). La idea central es desacoplar el tamaño de la cuadrícula del NCA de la resolución final, delegando el renderizado detallado a un decodifica-

dor ligero denominado *Local Pattern Producing Network* (LPPN). De este modo, el NCA mantiene una cuadrícula explícita y relativamente pequeña que evoluciona de manera local, mientras que la LPPN se encarga de producir el color en cualquier punto del espacio continuo a partir de información interpolada de las celdas vecinas.

En términos formales, para cada punto de muestreo p, la LPPN recibe dos entradas: (1) un estado celular promedio $s(p)$, obtenido mediante interpolación de las celdas cercanas en un primitivo geométrico, y (2) un vector de coordenadas locales $u(p)$, que codifica la posición relativa de p dentro de dicho primitivo. Dependiendo de la discretización empleada, estos primitivos pueden ser rectángulos, cubos, triángulos o polígonos generales, definidos por coordenadas $\lambda_i(p)$ que cumplen propiedades como unidad de partición y no negatividad. Para garantizar continuidad espacial, las coordenadas se transforman mediante funciones sinusoidales en cuadrículas cartesianas, o bien en mallas triangulares, obteniendo así campos posicionales suaves.

El resultado es un sistema que conserva las propiedades autoorganizativas de los NCA al tiempo que permite alcanzar mayor resolución sin un aumento proporcional del costo computacional. Desde una perspectiva biológica, el NCA puede interpretarse como un plano morfogenético que define la organización global, mientras que la LPPN añade los detalles finos sin necesidad de simular interacciones locales a gran escala. Este paralelismo, aunque principalmente ilustrativo, refuerza la idea de que los NCA pueden actuar como generadores de estructuras robustas y adaptables, mientras que módulos complementarios aportan precisión y escalabilidad.

A pesar de estas mejoras, una limitación persistente es que cada NCA debe entrenarse de manera específica para una textura o patrón dado. Incluso en escenarios donde múltiples NCA coexisten, cada modelo requiere su propio entrenamiento independiente. Esto ha motivado nuevas propuestas orientadas a la construcción de espacios compartidos de NCA, capaces de generalizar a múltiples texturas dentro de un mismo marco de entrenamiento.

3.5. Modelos generativos

A cada imagen generada con un NCA [14] le corresponde una ANN que está explícitamente entrenada para imitar ese comportamiento. Alejandro Hernández et al. [29] proponen una ANN de mayor capacidad que permita crecer múltiples imágenes, a lo que llamaron *Neural Cellular Automata Manifold* (NCAM).

El NCAM usa un modelo de convoluciones dinámicas para modelar un espacio de NCAs. Utiliza un auto-*encoder* que se compone de un codificador y un decodificador: El codificador transforma la entrada en una representación continua y luego categórica, que el decodificador reconstruye en el espacio original.

Un elemento destacable de la arquitectura propuesta es un parámetro entrenable llamado *Leak Factor* (LF) que mejora la estabilidad durante el entrenamiento. Este parámetro regula la cantidad de información que se filtra en cada paso del modelo, por ejemplo, un valor bajo retiene el conocimiento acumulado. Este valor se incrementa durante el entrenamiento a medida que mejora el rendimiento.

La arquitectura también emplea convoluciones dinámicas. Los *kernels* se generan a partir del *embedding* (incrustación) de cada instancia, permitiendo adaptabilidad sin modificar la estructura del NCA. El NCAM inicia desde una imagen *seed* y actualiza cada celda mediante percepciones locales (filtros Sobel + normalización) y capas densas por píxel, moduladas por el LF. Se exploran actualizaciones estocásticas y sincrónicas, siendo esta última más precisa y estable, tal como en [15].

Los experimentos confirman que la codificación y la actualización sincrónica mejoran la calidad de reconstrucción, incluso en ausencia de mecanismos de enmascaramiento. También se observa que aumentar la dimensionalidad del *embedding* mejora significativamente los resultados.

Por otro lado, Kevin Frans en [30] propone un modelo llamado *Conditional Neural Cellular Automata* (StampCA) cuya *seed* recibe un *embedding* el cual define la imagen en la cual va a crecer.

Dado un diseño deseado, una red de codificador lo comprime en un *embedding* el cual se utiliza como condición inicial en la NCA, que luego itera para crecer en la figura objetivo, esto permite crear nuevas configuraciones

Utilizando un *dataset*(conjunto de datos) con aproximadamente 3000 *emojis* de 32×32 píxeles fue capaz de entrenar a StampCA utilizando el mismo método de [14] para poder dibujar todos los *emojis* con una única NCA.

Para generar números de MNIST se adopta un enfoque tipo *Generative Adversarial Network* (GAN), donde el objetivo es crecer dígitos desde ruido aleatorio. Dado que entrenar directamente un NCA como generador GAN resultó inestable, se utilizó una estrategia en dos etapas: (1) Se entrena primero un generador tradicional (*feedforward*) para mapear vectores aleatorios a dígitos MNIST y (2) el NCA se entrena para imitar la salida de ese generador, logrando una generación más estable.

Este enfoque permite que el NCA genere dígitos sin necesidad de un *dataset* explícito, aprendiendo directamente a partir del objetivo GAN. Además, se observa un patrón de crecimiento distintivo: mientras los *emojis* crecen en ondas desde el centro, los dígitos MNIST se desarrollan siguiendo la forma del trazo.

Mingxiang Chen et al. [31] combinan los principios de *Variational Auto-Encoder* (VAE) con NCA. Utilizan un *encoder* que transforma una imagen en una distribución de probabilidad, las cuales forman un *embedding* el cual funciona como condición inicial del NCA. Posteriormente Rasmus Berg Palm et al. [32] nombran a este modelo como *Variational Neural Cellular Automata* (VNCA), quienes ademas aseguran que ninguno de los otros modelos vistos ([14], [29], [21], [23] y [30]) en realmente generativo.

El VNCA se basa en la arquitectura del modelo generativo probabilístico *Variational Auto-Encoder* (VAE) la cual aprende una distribución de probabilidad que codifica los datos observados. A partir de esta representación, el decodificador genera imágenes mediante un NCA que simula el crecimiento espacial desde una *seed*. A diferencia de VAE que decodifica la imagen directamente, el VNCA utiliza el *embedding* como configuración inicial de una cuadricula de celdas que evolucionan siguiendo las reglas definidas por una CNN.

Por otra parte, este modelo incorpora mecanismos de duplicación celular inspirado en la mitosis. Cada M pasos las celdas se duplican hacia la derecha y hacia abajo permitiendo a la imagen crecer de manera autoorganizada.

Los experimentos realizados en [32] demuestran que el VNCA es capaz de aprender procesos generativos autoorganizados tanto en dominios simples como MNIST, como en conjuntos más complejos como CelebA[7]. En MNIST, el modelo logra reconstrucciones precisas y muestras plausibles desde el espacio latente, con una log-verosimilitud de $\log p(x) \geq -84.23$ nats[8], su rendimiento es comparable a modelos autorregresivos como PixelCNN. Además, el análisis del espacio latente revela una estructura más separable y organizada que la de decodificadores convolucionales tradicionales, lo que sugiere una codificación más semántica y eficiente.

En CelebA, aunque las muestras generadas directamente no alcanzan calidad visual comparable a modelos de última generación, el VNCA muestra capacidad de reconstrucción robusta y mejora visual significativa al ajustar el parámetro β, que regula el equilibrio entre fidelidad de reconstrucción y calidad de generación.

[7] *Dataset* de imágenes de celebridades

[8] Unidad de medida utilizada en aprendizaje automático y estadística para expresar la log-verosimilitud o la entropía de un modelo

Finalmente, se explora la resiliencia del VNCA ante daño estructural durante el crecimiento. Al eliminar las capas de duplicación y entrenar con un esquema de recuperación de daño, el modelo aprende dinámicas estables capaces de restaurar imágenes deterioradas casi por completo, incluso tras múltiples pasos de evolución. Este comportamiento sugiere que el VNCA no solo codifica la imagen, sino que también es un proceso de reconstrucción en el estado celular.

3.6. Representaciones y aplicaciones adicionales

Hasta el momento, gran parte de los trabajos con NCA se han centrado en la morfogénesis y en su capacidad para generar estructuras visuales autoorganizadas. Sin embargo, también han surgido enfoques que amplían el marco conceptual hacia nuevas formas de representación y aplicaciones en dominios específicos. A continuación, se presentan algunas de estas variantes.

Mattie Tesfaldet et al. [33] extienden los NCA mediante mecanismos de atención inspirados en arquitecturas de *transformers*, dando lugar a *Vision Transformer Cellular Automata* (ViTCA). Esta arquitectura híbrida permite que las interacciones locales tengan acceso contextual a regiones más amplias a través de *self-attention*, lo que habilita organización global sin incurrir en la complejidad cuadrática típica de la atención completa. ViTCA adapta el proceso de los ViT tradicionales, que dividen una imagen en parches y los convierten en secuencias de *tokens*, al formato de una cuadrícula de celdas, donde cada celda almacena información visual, posicional y oculta. En lugar de aplicar atención global, restringe el cálculo a vecindarios locales, manteniendo así la naturaleza distribuida del sistema y reduciendo el costo computacional. El entrenamiento se realiza de forma iterativa, alternando entre inyectar entradas corruptas y actualizar parcialmente los estados celulares hasta reconstruir la imagen. De este modo, ViTCA combina eficiencia, capacidad de autoorganización y organización espacial coherente.

En un ámbito distinto, Jian Tang et al. [34] emplean CNN para simular el desarrollo de microestructuras en la solidificación de metales. Cada celda del dominio contiene un estado de fase $P \in [0, 1]$ (donde $p = 0$ representa líquido y $p = 1$ sólido), tres ángulos de Euler $\theta = (\alpha_1, \alpha_2, \alpha_3)$ que definen su orientación cristalográfica, y la magnitud de sub enfriamiento $\Delta T = T_{melting} - T$. La evolución se modela con un NCA que aplica recurrentemente una CNN para predecir los incrementos en fase, orientación y estados ocultos. Estos canales ocultos facilitan el aprendizaje de patrones temporales complejos sin necesidad de reglas explícitas. Con ello, el modelo reproduce fenómenos como crecimiento competitivo o direc-

ciones preferentes, con una eficiencia computacional sustancialmente mayor que la de muchos CA tradicionales y con capacidad de generalización fuera del rango de entrenamiento.

Por otro lado, Gordon H. Y. Li et al. [35] proponen el modelo *Photonic Neural Cellular Automata* (PNCA), que aprovecha la velocidad y la interconexión masiva de la fotónica para implementar NCA en hardware óptico. A diferencia de arquitecturas convencionales basadas en MLP o CNN, difíciles de trasladar a entornos fotónicos por sus conexiones densas, PNCA utiliza reglas locales aprendidas mediante técnicas de aprendizaje profundo. Esto permite realizar clasificación de imágenes en sistemas ópticos, donde la actualización celular se implementa mediante interferencia lineal de luz y activaciones ópticas no lineales. Los autores demuestran clasificación binaria con alta precisión utilizando un número muy reducido de parámetros fotónicos programables, mostrando además robustez frente a datos fuera de distribución. PNCA representa una vía prometedora para llevar el aprendizaje autoorganizado a plataformas fotónicas escalables.

En el ámbito de la biología computacional, Caitlin Grasso y Josh Bongard [36] introducen los *Empowered Neural Cellular Automata* (ENCA), una extensión que incorpora el concepto de *empowerment* —la cantidad de control que una célula puede ejercer sobre su entorno— formalizado mediante medidas de información mutua entre acciones pasadas y estados sensoriales futuros. Incluir *empowerment* como objetivo adicional en tareas de morfogénesis mejora la aptitud evolutiva y produce patrones de crecimiento más robustos y adaptativos, posicionando a cada célula como un agente autónomo dentro de un sistema distribuido y abriendo vías para estudiar coordinación y comunicación celular desde una óptica computacional.

Finalmente, mientras que modelos como ViTCA, PNCA o ENCA amplían las capacidades arquitectónicas y las plataformas de aplicación de los NCA, Richardson et al. [37] utilizan los NCA como herramienta para modelar fenómenos dinámicos complejos. Entrenando NCA mediante optimización basada en gradientes para reproducir trayectorias espacio-temporales generadas por ecuaciones diferenciales parciales (por ejemplo, Gray–Scott), este enfoque captura tanto estructuras transitorias como patrones estacionarios —incluyendo la formación de patrones tipo Turing— y aborda el problema inverso de inferir reglas locales a partir de comportamiento emergente. Con una complejidad de red mínima, los autores muestran capacidad de generalización, respeto por simetrías y robustez frente a perturbaciones, posicionando a los NCA como un marco explicativo y eficiente para sistemas biológicos y físicos.

Conclusiones del estado del arte

Se observa que los NCA han evolucionado desde sus primeras aplicaciones en morfogénesis, donde se exploró su capacidad de generar estructuras complejas partiendo de una única célula activa, hacia modelos que buscan una mayor versatilidad generativa, capaces de producir dinámicas visuales, texturas, comportamientos adaptativos e incluso manipular estrategias, muchos de ellos inspirados en la definición de Mordvintsev. Además, se han desarrollado arquitecturas híbridas que no generan un NCA individual, sino que modelan un espacio de NCA, así como configuraciones generativas y bioinspiradas.

En conjunto, estos trabajos evidencian que los NCA no solo constituyen un marco computacional de interés teórico, sino también una herramienta con aplicaciones en inteligencia artificial, ciencias naturales e ingeniería. Esta diversidad de enfoques abre la puerta a explorar nuevas adaptaciones y propuestas, especialmente en áreas de aprendizaje profundo, autoorganización y simulación de procesos dinámicos.

Bibliografía

[1] J. Damerow. (2025) John von neumann's cellular automata. Accedido el 23 de julio de 2025. [Online]. Available: https://embryo.asu.edu/pages/john-von-neumanns-cellular-automata

[2] G. J. Martínez. (2025) Teoría del campo promedio en autómatas celulares similares a "the game of life".

[3] K. C. II and S. Yu, "Undecidability of ca classification schemes," *Complex Systems*, vol. 2, pp. 177–190, 1988.

[4] brgfx, "Freepik graphic resource," http://www.freepik.com, designed by brgfx / Freepik. Accessed: 2025-09-17.

[5] S. Wolfram, "Cellular automata as simple self-organizing systems," Caltech, Tech. Rep. CALT-68-938, 1982, general Theoretical Physics, 11 páginas.

[6] E. W. Weisstein, "Cellular automaton," https://mathworld.wolfram.com/CellularAutomaton.html, 2025, from MathWorld–A Wolfram Resource, accedido el 23 de julio de 2025.

[7] Fundación BBVA, "Diccionario - fundación bbva," https://www.fbbva.es/diccionario/retcula, 2025, accedido el 25 de julio de 2025.

[8] D. Grattarola, L. Livi, and C. Alippi, "Learning graph cellular automata," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 20 983–20 994. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/af87f7cdcda223c41c3f3ef05a3aaeea-Paper.pdf

[9] P. Gács, "Probabilistic cellular automata with andrei toom," *Brazilian Journal of Probability and Statistics*, vol. 38, no. 2, Jun. 2024, accedido el 23 de julio de 2025. [Online]. Available: https://doi.org/10.1214/24-bjps605

[10] H. Hartman, P. Tamayo, and W. Klein, "Inhomogeneous cellular automata and statistical mechanics," *Complex Systems*, vol. 1, no. 2, 1987, massachusetts Institute of Technology and Boston University. [Online]. Available: https://content.wolfram.com/uploads/sites/13/2018/02/01-2-2.pdf

[11] S. Wolfram, *A New Kind of Science*. Champaign, IL: Wolfram Media, 2002.

[12] L. Becker. (2025) Continuous cellular automata – hegl. HEGL – Heidelberg Experimental Geometry Lab, accedido el 23 de julio de 2025. [Online]. Available: https://hegl.mathi.uni-heidelberg.de/continuous-cellular-automata

[13] W. Gilpin, "Cellular automata as convolutional neural networks," *Physical Review E*, vol. 100, no. 3, Sep. 2019, accedido el 29 de julio de 2025. [Online]. Available: https://doi.org/10.1103/PhysRevE.100.032402

[14] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, "Growing neural cellular automata," *Distill*, 2020, https://distill.pub/2020/growing-ca.

[15] *Asynchronicity in Neural Cellular Automata*, ser. ALIFE 2022: The 2022 Conference on Artificial Life, vol. ALIFE 2021: The 2021 Conference on Artificial Life, 07 2021. [Online]. Available: https://doi.org/10.1162/isal_a_00461

[16] A. Mordvintsev, E. Randazzo, and C. Fouts, "Growing isotropic neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2205.01681

[17] *Emergent Multiscale Structures and Generative Potential of Isotropic Neural Cellular Automata*, ser. ALIFE 2022: The 2022 Conference on Artificial Life, vol. ALIFE 2024: Proceedings of the 2024 Artificial Life Conference, 07 2024. [Online]. Available: https://doi.org/10.1162/isal_a_00745

[18] E. Randazzo, A. Mordvintsev, and C. Fouts, "Growing steerable neural cellular automata," 2023. [Online]. Available: https://arxiv.org/abs/2302.10197

[19] E. Randazzo, A. Mordvintsev, E. Niklasson, M. Levin, and S. Greydanus, "Self-classifying mnist digits," *Distill*, 2020, https://distill.pub/2020/selforg/mnist.

[20] E. Randazzo, A. Mordvintsev, E. Niklasson, and M. Levin, "Adversarial reprogramming of neural cellular automata," *Distill*, 2021, https://distill.pub/selforg/2021/adversarial.

[21] D. Zhang, C. Choi, J. Kim, and Y. M. Kim, "Learning to generate 3d shapes with generative cellular automata," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=rABUmU3ulQh

[22] S. Sudhakaran, D. Grbic, S. Li, A. Katona, E. Najarro, C. Glanois, and S. Risi, "Growing 3d artefacts and functional machines with neural cellular automata," 2021. [Online]. Available: https://arxiv.org/abs/2103.08737

[23] K. Horibe, K. Walker, and S. Risi, "Regenerating soft robots through neural cellular automata," 2021. [Online]. Available: https://arxiv.org/abs/2102.02579

[24] E. Niklasson, A. Mordvintsev, E. Randazzo, and M. Levin, "Self-organising textures," *Distill*, 2021, https://distill.pub/selforg/2021/textures.

[25] E. R. Alexander Mordvintsev, Eyvind Niklasson, "Texture generation with neural cellular automata," *CoRR*, vol. abs/2105.07299, 2021. [Online]. Available: https://arxiv.org/abs/2105.07299

[26] A. Mordvintsev and E. Niklasson, "μnca: Texture generation with ultra-compact neural cellular automata," *CoRR*, vol. abs/2111.13545, 2021. [Online]. Available: https://arxiv.org/abs/2111.13545

[27] E. Pajouheshgar, Y. Xu, A. Mordvintsev, E. Niklasson, T. Zhang, and S. Süsstrunk, "Mesh neural cellular automata," 2024. [Online]. Available: https://arxiv.org/abs/2311.02820

[28] E. Pajouheshgar, Y. Xu, A. Abbasi, A. Mordvintsev, W. Jakob, and S. Süsstrunk, "Neural cellular automata: From cells to pixels," 2025. [Online]. Available: https://arxiv.org/abs/2506.22899

[29] A. H. Ruiz, A. Vilalta, and F. Moreno-Noguer, "Neural cellular automata manifold," 2021. [Online]. Available: https://arxiv.org/abs/2006.12155

[30] K. Frans, "Stampca: Growing emoji with conditional neural cellular automata," *kvfrans.com*, 2021. [Online]. Available: http://kvfrans.com/stampca-conditional-neural-cellular-automata/

[31] M. Chen and Z. Wang, "Image generation with neural cellular automatas," 2020. [Online]. Available: https://arxiv.org/abs/2010.04949

[32] R. B. Palm, M. González-Duque, S. Sudhakaran, and S. Risi, "Variational neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2201.12360

[33] M. Tesfaldet, D. Nowrouzezahrai, and C. Pal, "Attention-based neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2211.01233

[34] J. Tang, S. Kumar, L. De Lorenzis, and E. Hosseini, "Neural cellular automata for solidification microstructure modelling," *Computer Methods in Applied Mechanics and Engineering*, vol. 414, p. 116197, Sep. 2023. [Online]. Available: http://dx.doi.org/10.1016/j.cma.2023.116197

[35] G. Li, C. Leefmans, J. Williams *et al.*, "Deep learning with photonic neural cellular automata," *Light: Science & Applications*, vol. 13, no. 283, 2024, accedido el 29 de julio de 2025. [Online]. Available: https://doi.org/10.1038/s41377-024-01651-7

[36] C. Grasso and J. Bongard, "Empowered neural cellular automata," ser. GECCO '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 108–111. [Online]. Available: https://doi.org/10.1145/3520304.3529067

[37] A. D. Richardson, T. Antal, R. A. Blythe, and L. J. Schumacher, "Learning spatio-temporal patterns with neural cellular automata," 2024. [Online]. Available: https://arxiv.org/abs/2310.14809